

Отчёт по лабораторной работе №6

Компьютерный практикум по статистическому анализу

Надежда Александровна Рогожина

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
4.1	Примеры	8
4.2	Задания для самостоятельного выполнения	13
5	Выводы	25
	Список литературы	26

Список иллюстраций

4.1	Импорт библиотек	8
4.2	Модель экспоненциального роста	9
4.3	Модель экспоненциального роста. График	9
4.4	Модель экспоненциального роста	9
4.5	Модель экспоненциального роста. График	10
4.6	Система Лоренца	10
4.7	Система Лоренца	10
4.8	Система Лоренца. График	11
4.9	Система Лоренца. График	11
4.10	Модель Лотки-Вольтерры	11
4.11	Модель Лотки-Вольтерры	12
4.12	Модель Лотки-Вольтерры. График	12
4.13	Модель Лотки-Вольтерры. Фазовый портрет	12
4.14	Решение	13
4.15	График	14
4.16	Приближение	14
4.17	Анимация	14
4.18	Решение	15
4.19	График	16
4.20	Решение + график	17
4.21	Анимация	17
4.22	Решение + график	18
4.23	Решение + график	19
4.24	Фазовый портрет	19
4.25	Решение + график	20
4.26	Анимация	20
4.27	Решение + график	21
4.28	Анимация	21
4.29	Фазовый портрет	22
4.30	Решение + график	23
4.31	Анимация	23
4.32	Фазовый портрет	24

Список таблиц

1 Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 6.2.
2. Выполните задания для самостоятельной работы (раздел 6.4).

3 Теоретическое введение

Научные вычисления традиционно требуют высочайшей производительности, однако эксперты по доменам в значительной степени перенесены на более медленные динамические языки для ежедневной работы. К счастью, методы современного языка и компилятора позволяют в основном устранить компромисс производительности и обеспечивать достаточно продуктивную среду для прототипирования и достаточно эффективно для развертывания применений, интенсивных производительности. Язык программирования Julia заполняет эту роль: это гибкий динамический язык, подходящий для научных и численных вычислений, с производительностью, сравнимой с традиционными статичными языками.

4 Выполнение лабораторной работы

4.1 Примеры

Первым делом импортируем нужные библиотеки (рис. 4.1):

```
In [1]: # подключаем необходимые пакеты:
import Pkg
Pkg.add("DifferentialEquations")
using DifferentialEquations

Updating registry at 'C:\Users\Home\Julia\registries\General.toml'
Resolving package versions...
No changes to 'C:\Users\Home\Julia\environments\v1.11\Project.toml'
No changes to 'C:\Users\Home\Julia\environments\v1.11\Manifest.toml'
Precompiling project...
19614.8 ms ✓ OrdinaryDiffEqNonlinearSolve
48269.9 ms ✓ BoundaryValueDiffEq
31700.0 ms ✓ OrdinaryDiffEqStabilizedIRK
34458.9 ms ✓ OrdinaryDiffEqPDIRK
38551.5 ms ✓ OrdinaryDiffEqSDIRK
47324.0 ms ✓ StochasticDiffEq
20336.1 ms ✓ OrdinaryDiffEqIMEXMultistep
48381.3 ms ✓ OrdinaryDiffEqBDF
66814.6 ms ✓ OrdinaryDiffEqIRK
64221.4 ms ✓ OrdinaryDiffEqDefault
11587.9 ms ✓ OrdinaryDiffEq
12023.3 ms ✓ DelayDiffEq
24563.7 ms ✓ DifferentialEquations
13 dependencies successfully precompiled in 326 seconds. 494 already precompiled.

In [2]: # подключаем необходимые пакеты:
Pkg.add("Plots")
using Plots

Resolving package versions...
No changes to 'C:\Users\Home\Julia\environments\v1.11\Project.toml'
No changes to 'C:\Users\Home\Julia\environments\v1.11\Manifest.toml'
```

Рисунок 4.1: Импорт библиотек

Далее, рассмотрим примеры решения модели экспоненциального роста (рис. 4.2, рис. 4.3, рис. 4.4, рис. 4.5), решение системы Лоренца (рис. 4.6, рис. 4.7, рис. 4.8, рис. 4.9), решение модели Лотки-Вольтерры (рис. 4.10, рис. 4.11, рис. 4.12, рис. 4.13).


```
In [9]: # задаём описание модели с начальными условиями:
a = 0.98
f(u,p,t) = a*u
u0 = 1.0

# задаём интервал времени:
tspan = (0.0,1.0)

prob = ODEProblem(f,u0,tspan)
sol = solve(prob)
```

```
Out[9]: retcode: Success
Interpolation: 3rd order Hermite
t: 5-element Vector{Float64}:
 0.0
 0.10042494449239292
 0.3521860297865888
 0.6934436122197829
 1.0
u: 5-element Vector{Float64}:
 1.0
 1.1034222047865465
 1.4121908713484919
 1.9730384457359198
 2.6644561424814266
```

Рисунок 4.2: Модель экспоненциального роста

```
In [5]: # строим графики:
plot(sol, linewidth=3, title="Модель экспоненциального роста", xaxis="Время", yaxis="u(t)", label="u(t)", color="black")
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, label="Аналитическое решение")
```

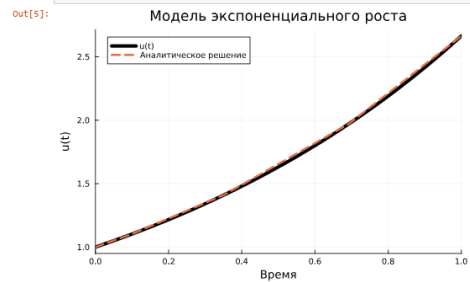


Рисунок 4.3: Модель экспоненциального роста. График

```
In [11]: sol = solve(prob, abstol=1e-8, reltol=1e-8)
```

```
Out[11]: retcode: Success
Interpolation: 3rd order Hermite
t: 9-element Vector{Float64}:
 0.0
 0.04127492324135852
 0.14679917457239627
 0.2863153763968216
 0.43819404263820116
 0.6118923088547421
 0.7985657481708535
 0.9993514410032294
 1.0
u: 9-element Vector{Float64}:
 1.0
 1.0412786454705882
 1.1547261208857074
 1.3239094565289573
 1.5363817850042807
 1.8214892991042442
 2.187139297776232
 2.6627632840763393
 2.6644562419335163
```

Рисунок 4.4: Модель экспоненциального роста

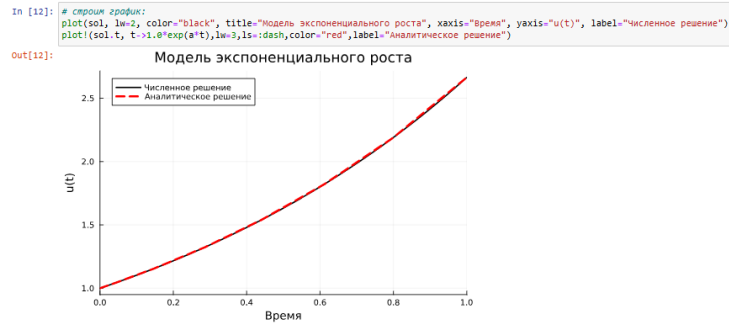


Рисунок 4.5: Модель экспоненциального роста. График

```
In [13]: # задаём описание модели:
function lorenz!(du,u,p,t)
    σ,ρ,β = p
    du[1] = σ*(u[2]-u[1])
    du[2] = u[1]*(ρ-u[3]) - u[2]
    du[3] = u[1]*u[2] - β*u[3]
end
```

Out[13]: lorenz! (generic function with 1 method)

Рисунок 4.6: Система Лоренца

```
In [14]: # задаём начальные условия:
u0 = [1.0, 0.0, 0.0]

# задаём значения параметров:
p = (10, 28, 8/3)

# задаём интервал времени:
tspan = (0.0, 100.0)

# решение:
prob = ODEProblem(lorenz!, u0, tspan, p)
sol = solve(prob)
```

Out[14]: retcode: Success
Interpolation: 3rd order Hermite
t: 1292-element Vector{Float64}:
0.0
3.5678604836301404e-5
0.0003924646531993154
0.003262408731175873
0.009058076622686189
0.01695647090176743
0.027689960116428883
0.041856352219618344
0.060240411865493296
0.08368541210909924
0.11336499336103911
0.14862181393426632
0.1870397836913972
⋮
99.25245983911758
99.3123010170333
99.37100059024901
99.43545175575305
99.50217600300971
99.56297541572351
99.62622492183432
99.69561088424294
99.77387244562912
99.86354266863755
99.93826978910452
100.0
u: 1292-element Vector{Vector{Float64}}:
[1.0, 0.0, 0.0]
[0.9996434557625105, 0.0009988049817849058, 1.781434788799189e-8]
[0.9961045497425811, 0.010965399721242457, 2.1469553658389193e-6]

Рисунок 4.7: Система Лоренца

```
In [25]: pyplot()
plot(sol, vars=(1,2,3), lw=0.5, title="Аттрактор Лоренца", xaxis="x", yaxis="y", zaxis="z", legend=false, size=(800, 500))
Out[25]:
```

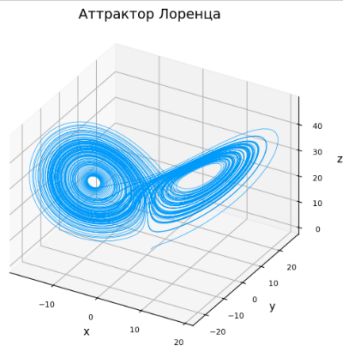


Рисунок 4.8: Система Лоренца. График

```
In [27]: plot(sol, vars=(1,2,3), denseplot=false, lw=0.5, title="Аттрактор Лоренца", xaxis="x", yaxis="y", zaxis="z",
legend=false, size=(800, 500))
Out[27]:
```

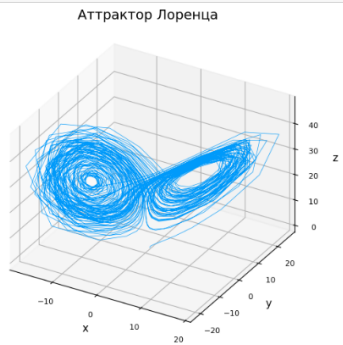


Рисунок 4.9: Система Лоренца. График

```
In [28]: import Pkg
Pkg.add("ParameterizedFunctions")
using ParameterizedFunctions

This dependencies successfully precompiled in 1211 seconds. Was already precompiled.
dependencies precompiled but different versions are currently loaded. Restart Julia to access the new versions. Otherwise
e, loading dependencies of these packages may trigger further precompilation to work with the unexpected versions.
dependency had output during precompilation:
SCCNonlinearSolve
[pid 25392] waiting for IO to finish:
Handle type      uv_handle_t->data
fs_event         0000022609913578->00000226d7cc3a0
timer            00000226e1074a00->00000226d7cc3d0
This means that a package has started a background task or event source that has not finished running. For precompilation
to complete successfully, the event source needs to be closed explicitly. See the developer documentation on fixing precompil
ation hangs for more help.

[ Info: Precompiling ParameterizedFunctions [65888b18-ceab-5608-b2b9-181511a3b968] (cache misses: wrong dep version loaded
(2))
[ Info: Precompiling SymbolicsPreallocationToolsExt [d479e226-fb54-5e8e-a75e-a7af7f39127f] (cache misses: wrong dep version l
oaded (2))
[ Info: Precompiling BlockArraysBandedMatricesExt [231a03ec-9455-50cf-8098-89f612a0b043] (cache misses: wrong dep version loa
ded (2))

In [29]: # задаём описание модели:
Iv1 = @ode_def Lotkavolterra begin
dx = a*x - b*x*y
dy = -c*y + d*x*y
end a b c d

Warning: Independent variable t should be defined with @independent_variables t.
@ ModelingToolkit C:\Users\Home\Julia\packages\ModelingToolkit\src\utils.jl:1119

Out[29]: (::Lotkavolterra{var####ParameterizedDiffEqFunction#275", var####ParameterizedTGradFunction#276", var####ParameterizedJacobia
nFunction#277", Nothing, Nothing, ODESystem}) (generic function with 1 method)
```

Рисунок 4.10: Модель Лотки-Вольтерры

```
In [30]: # задаём начальные условия:
u0 = [1.0, 1.0]
# задаём значения параметров:
p = (1.5, 1.0, 3.0, 1.0)
# задаём интервал времени:
tspan = (0.0, 10.0)

prob = ODEProblem(lv!,u0,tspan,p)
sol = solve(prob)

Out[30]: retcode: Success
Interpolation: 3rd order Hermite
t: 34-element Vector{Float64}:
 0.0
 0.0776004743154256
 0.2326451370670694
 0.42911851563726466
 0.679082199936080
 0.9444046279774128
 1.2674601918628516
 1.61929140093895
 1.9869755481782074
 2.2640903679981617
 2.5125406833621076
 2.746028291948325
 3.03006676853378
 3.3125406833621076
 3.5869755481782074
 3.8600903679981617
 4.1325406833621076
 4.406028291948325
 4.67914366165145
 4.95164366165145
 5.22364366165145
 5.49614366165145
 5.76864366165145
 6.04114366165145
 6.31364366165145
 6.58614366165145
 6.85864366165145
 7.13114366165145
 7.40364366165145
 7.67614366165145
 7.94864366165145
 8.22114366165145
 8.49364366165145
 8.76614366165145
 9.03864366165145
 9.31114366165145
 9.58364366165145
 9.85614366165145
10.0
u: 34-element Vector{Vector{Float64}}:
 [1.0, 1.0]
 [1.0454942346944578, 0.8576684823217128]
```

Рисунок 4.11: Модель Лотки-Вольтерры

```
In [31]: plot(sol, label = ["Жертвы" "хищники"], color="black", ls=[:solid :dash], title="Модель Лотки- Вольтерры",
          хaxis="Время",уaxis="Размер популяции")

Out[31]:
```

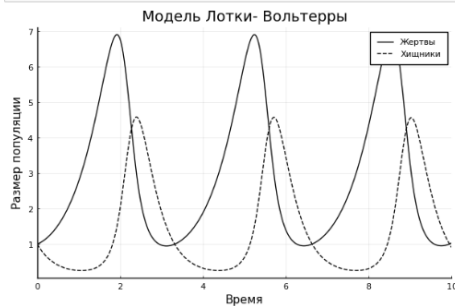


Рисунок 4.12: Модель Лотки-Вольтерры. График

```
In [32]: plot(sol,vars=(1,2), color="black", хaxis="жертвы", уaxis="хищники", legend=false)

Out[32]:
```

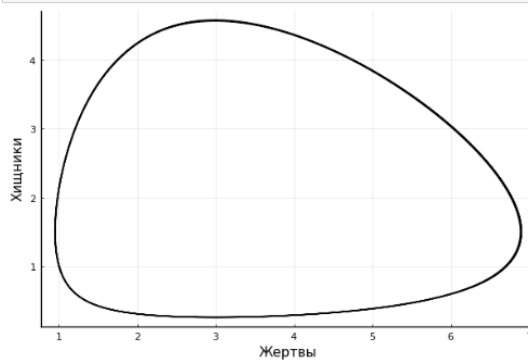


Рисунок 4.13: Модель Лотки-Вольтерры. Фазовый портрет

4.2 Задания для самостоятельного выполнения

1. Реализовать и проанализировать модель роста численности изолированной популяции (модель Мальтуса, рис. 4.14, рис. 4.15, рис. 4.16, рис. 4.17):

$$\dot{x} = ax, a = b - c,$$

где $x(t)$ — численность изолированной популяции в момент времени t , a — коэффициент роста популяции, b — коэффициент рождаемости, c — коэффициент смертности. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

```
In [62]: # задаём описание модели:
function lorenz!(du,u,p,t)
    b, c = p
    du[1] = (b-c)*u[1]
end
# задаём начальное условие:
u0 = [146150789]
# задаём значения параметров:
p = (2.5, 2.7)
# задаём интервал времени:
tspan = (0.0, 100.0)
# решение:
prob = ODEProblem(lorenz!, u0, tspan, p)
sol = solve(prob)

Out[62]: retcode: Success
Interpolation: 3rd order Hermite
t: 33-element Vector{Float64}:
 0.0
 0.13797296614631024
 0.3633680539606021
 2.365187749977535
 3.9989569299631347
 6.017945962851735
 8.299536578074452
10.874489879371303
13.674528574327505
16.686540430053498
19.867647860571488
23.195537295617157
26.64163223552912
...
60.42552491997608
64.3203175158615
68.2259482592793
72.140199491089965
76.06130749820188
79.98786661525509
83.91876222676552
87.85311281213939
91.790231168599
95.72959936041022
99.6708606580165
100.0
u: 33-element Vector{Vector{Float64}}:
 [1.46150789e8]
```

Рисунок 4.14: Решение

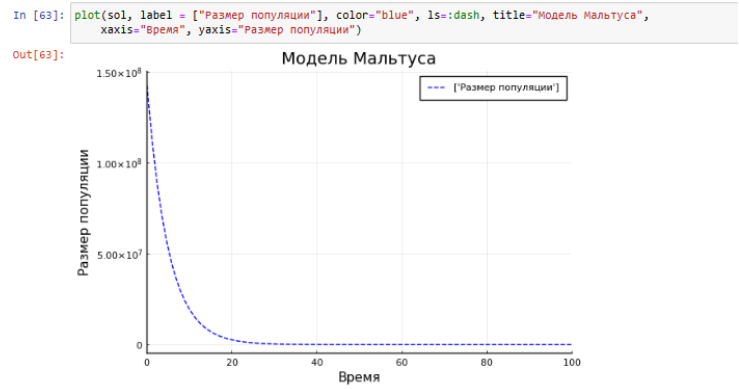


Рисунок 4.15: График

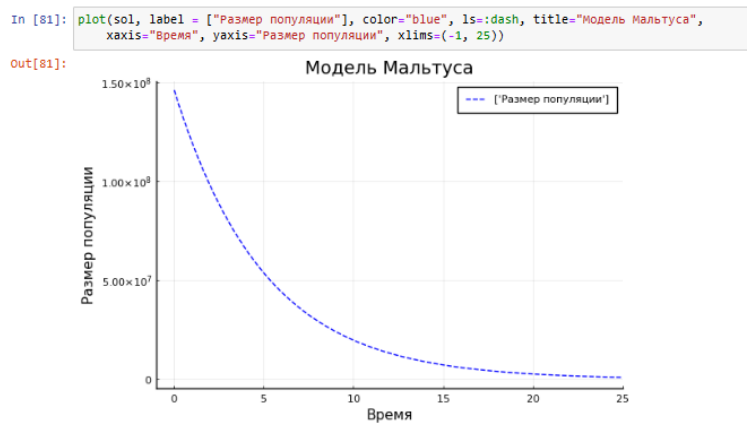


Рисунок 4.16: Приближение

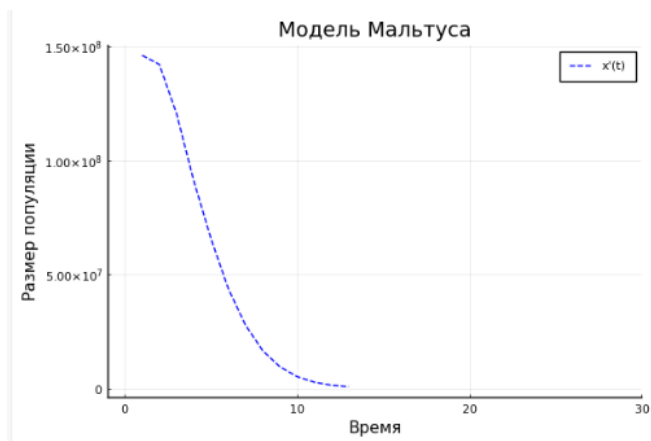


Рисунок 4.17: Анимация

2. Реализовать и проанализировать логистическую модель роста популяции (рис. 4.18, рис. 4.19), заданную уравнением:

$$\dot{x} = rx \left(1 - \frac{x}{k}\right), r > 0, k > 0,$$

r — коэффициент роста популяции, k — потенциальная ёмкость экологической системы (предельное значение численности популяции). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

```
In [93]: # задаём описание модели:
function lorenz!(du,u,p,t)
    r, k = p
    du[1] = r*u[1] * (1 - u[1]/k)
end
# задаём начальное условие:
u0 = [1000]
# задаём значения параметров:
p = (1.7, 1000000)
# задаём интервал времени:
tspan = (0.0, 10.0)
# решение:
prob = ODEProblem(lorenz!, u0, tspan, p)
sol = solve(prob)

Out[93]: retcode: Success
Interpolation: 3rd order Hermite
t: 20-element Vector{Float64}:
 0.0
 0.08092490211072892
 0.24273377752547187
 0.44728385662754906
 0.698690888827799
 0.947891699540147
 1.334543162855148
 1.7154191500254026
 2.135165102478015
 2.592051114188237
 3.0864681050897325
 3.622304155764195
 4.215969483897827
 5.051913541318896
 5.706051453567081
 6.4375373280696764
 7.170224237385194
 8.051592067940058
 9.051111975931533
10.0
u: 20-element Vector{Vector{Float64}}:
 [1000.0]
 [1147.315493276144]
```

Рисунок 4.18: Решение

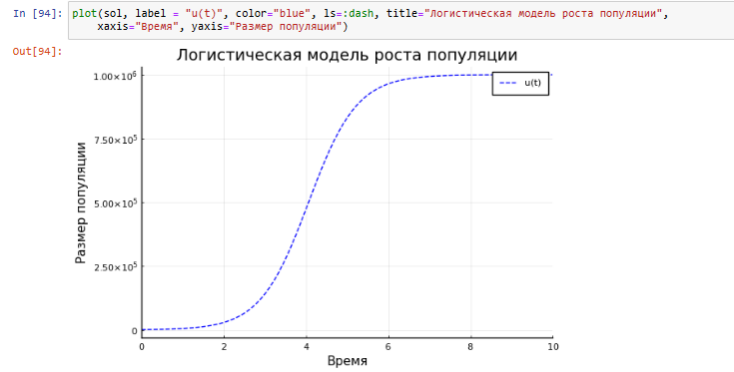


Рисунок 4.19: График

3. Реализовать и проанализировать модель эпидемии Кермака–Маккендрика (SIR-модель, рис. 4.20, рис. 4.21):

$$\begin{cases} \dot{s} = -\beta i s \\ \dot{i} = \beta i s - v i \\ \dot{r} = v i \end{cases}$$

где $s(t)$ — численность восприимчивых к болезни индивидов в момент времени t , $i(t)$ — численность инфицированных индивидов в момент времени t , $r(t)$ — численность переболевших индивидов в момент времени t , β — коэффициент интенсивности контактов индивидов с последующим инфицированием, v — коэффициент интенсивности выздоровления инфицированных индивидов. Численность популяции считается постоянной, т.е. $\dot{s} + \dot{i} + \dot{r} = 0$. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией).

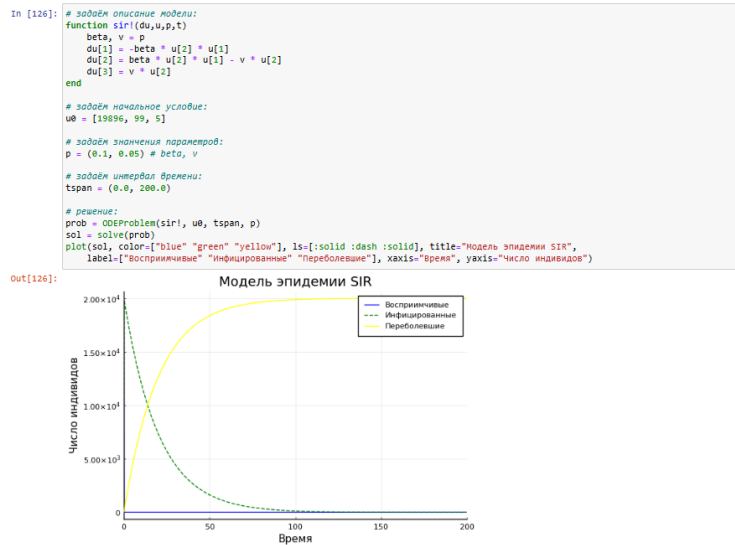


Рисунок 4.20: Решение + график

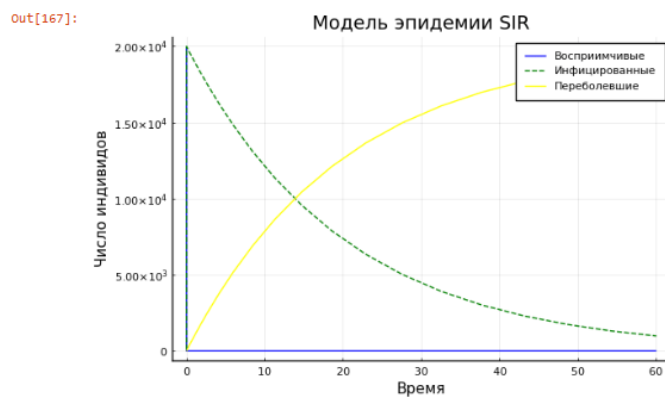


Рисунок 4.21: Анимация

4. Как расширение модели SIR (Susceptible-Infected-Removed) по результатам эпидемии испанки была предложена модель SEIR (Susceptible-Exposed-

Infected-Removed, рис. 4.22):

$$\begin{cases} \dot{s}(t) = -\frac{\beta}{N}i(t)s(t) \\ \dot{e}(t) = \frac{\beta}{N}i(t)s(t) - \delta e(t) \\ \dot{i}(t) = \delta e(t) - \gamma i(t) \\ \dot{r} = \gamma i(t) \end{cases}$$

Размер популяции сохраняется:

$$s(t) + e(t) + i(t) + r(t) = N.$$

Исследуйте, сравните с SIR.

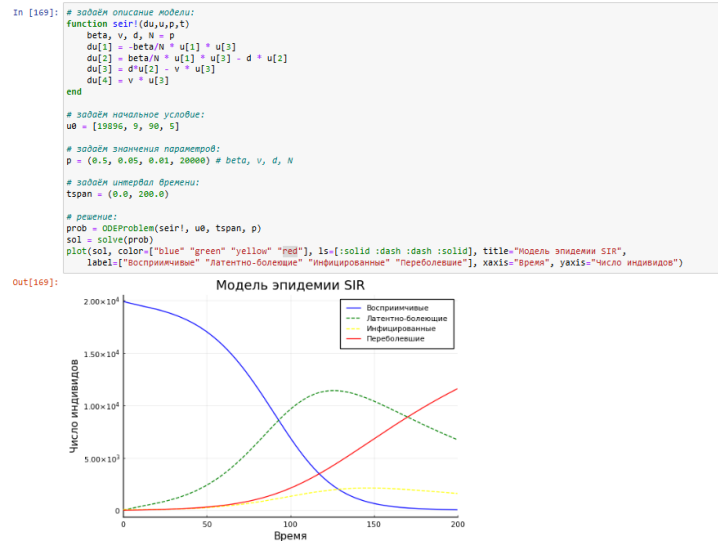


Рисунок 4.22: Решение + график

5. Для дискретной модели Лотки–Вольтерры (рис. 4.23, рис. 4.24):

$$\begin{cases} X_1(t+1) = aX_1(t)(1 - X_1(t)) - X_1(t)X_2(t) \\ X_2(t+1) = -cX_2(t) + dX_1(t)X_2(t) \end{cases}$$

с начальными данными $a = 2$, $c = 1$, $d = 5$ найдите точку равновесия. Получите и сравните аналитическое и численное решения. Численное решение изобразите на фазовом портрете.

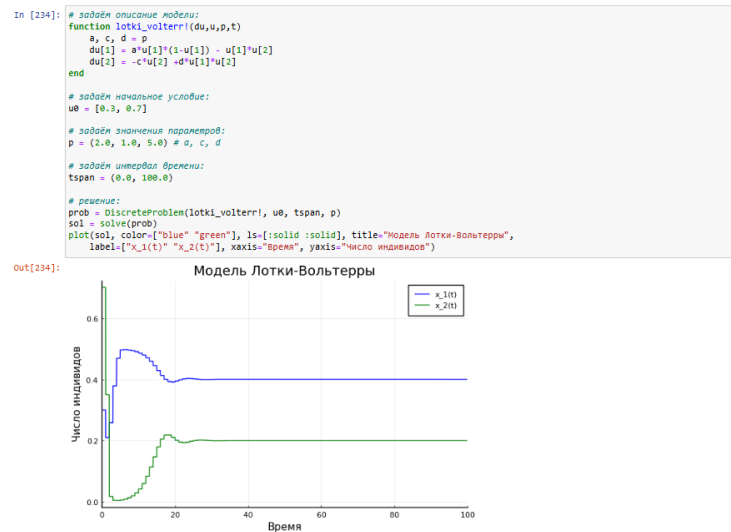


Рисунок 4.23: Решение + график

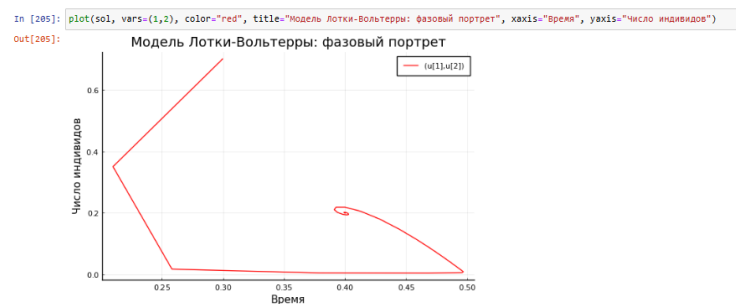


Рисунок 4.24: Фазовый портрет

6. Реализовать на языке Julia модель отбора на основе конкурентных отношений (рис. 4.25, рис. 4.26):

$$\begin{cases} \dot{x} = \alpha x - \beta xy \\ \dot{y} = \alpha y - \beta xy \end{cases}$$

Начальные данные и параметры задать самостоятельно и пояснить их выбор.

Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

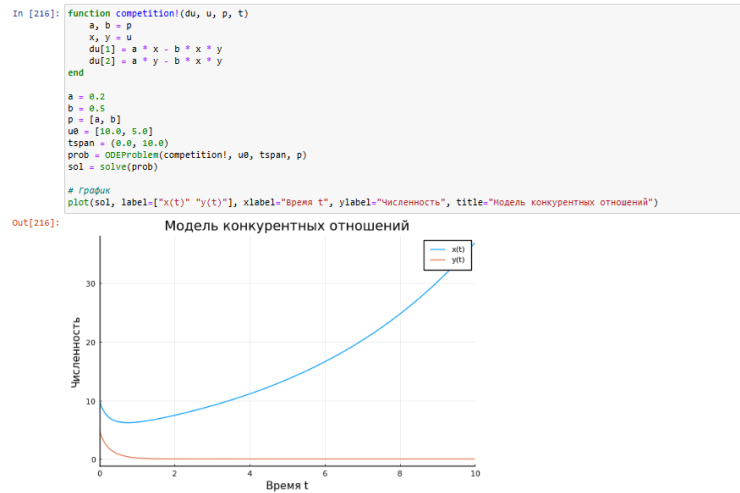


Рисунок 4.25: Решение + график



Рисунок 4.26: Анимация

7. Реализовать на языке Julia модель консервативного гармонического осциллятора (рис. 4.27, рис. 4.28, рис. 4.29):

$$\ddot{x} + \omega_0^2 x = 0, x(t_0) = x_0, \dot{x}(t_0) = y_0,$$

где ω_0 — циклическая частота. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе

с анимацией) и фазовый портрет.

Модель консервативного гармонического осциллятора

```
In [222]: function oscillator_conservative!(du, u, p, t)
          w0 = p[1]
          x, v = u
          du[1] = v
          du[2] = -w0^2 * x
        end

          p = [1.0]
          u0 = [1.0, 0.0]
          tspan = (0.0, 10.0)
          prob = ODEProblem(oscillator_conservative!, u0, tspan, p)
          sol = solve(prob)

          plot(sol, label=["x(t)" "v(t)"], xlabel="Время t", ylabel="Значение", title="Консервативный осциллятор")
```

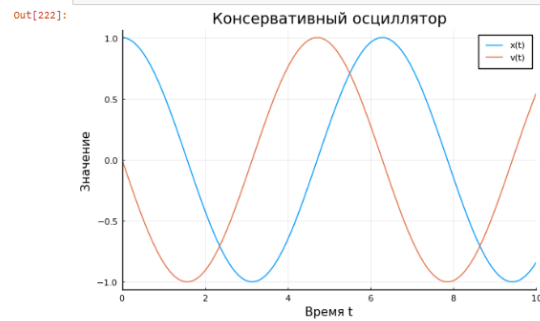


Рисунок 4.27: Решение + график

```
In [223]: # Animation
          anim = @animate for t in 0:0.1:10
            plot(sol, tspan=(0, t), label=["x(t)" "v(t)"], xlabel="Время t", ylabel="Значение",
                  title="Консервативный осциллятор (анимация)", xlim=(0,10), ylim=(-2,2))
          end
          gif(anim, "oscillator_conservative_animation.gif", fps=10)

          [ Info: Saved animation to C:\Users\HOME\Documents\РУДН\Компьютерный практикум по статистическому анализу\lab\oscillator_conse
rnative_animation.gif]
```

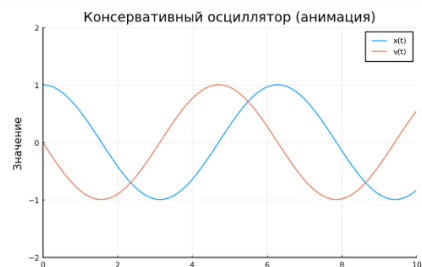


Рисунок 4.28: Анимация

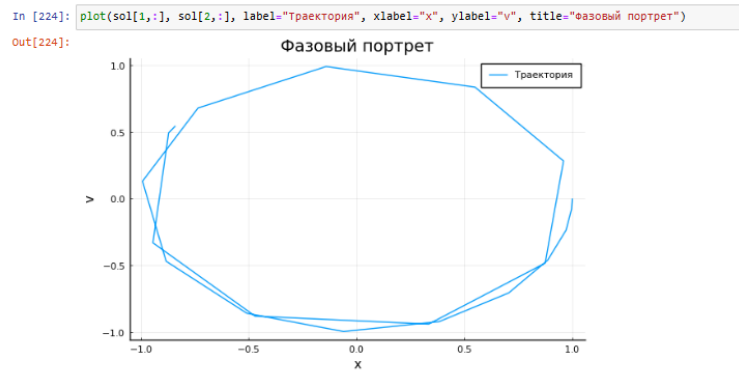


Рисунок 4.29: Фазовый портрет

8. Реализовать на языке Julia модель свободных колебаний гармонического осциллятора (рис. 4.30, рис. 4.31, рис. 4.32):

$$\ddot{x} + 2\gamma\dot{x} + \omega_0^2 x = 0, x(t_0) = x_0, \dot{x}(t_0) = y_0,$$

где ω_0 — циклическая частота, γ — параметр, характеризующий потери энергии. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

Свободные колебания гармонического осциллятора

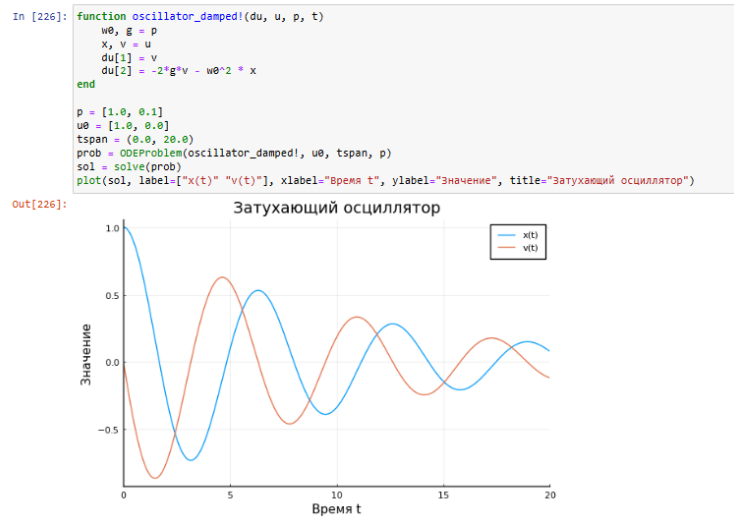


Рисунок 4.30: Решение + график

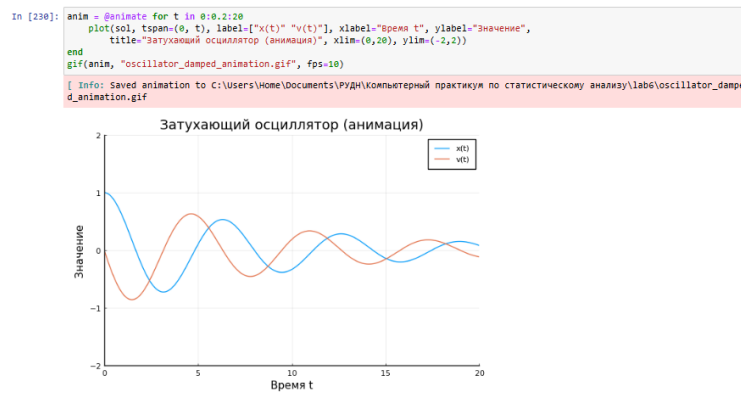


Рисунок 4.31: Анимация

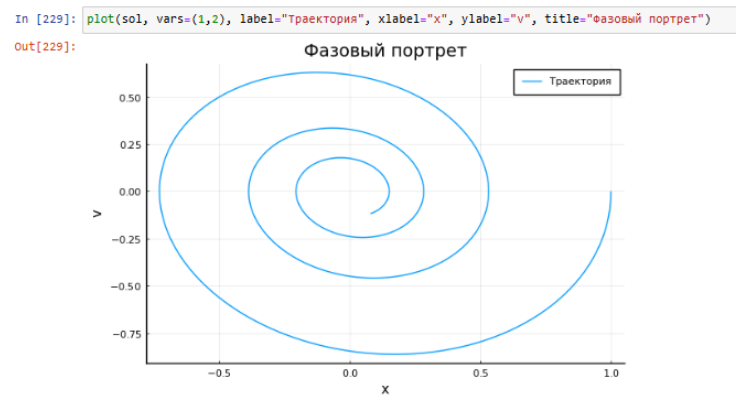


Рисунок 4.32: Фазовый портрет

5 Выводы

В ходе работы были изучены возможности Julia в решении дифференциальных уравнений.

Список литературы